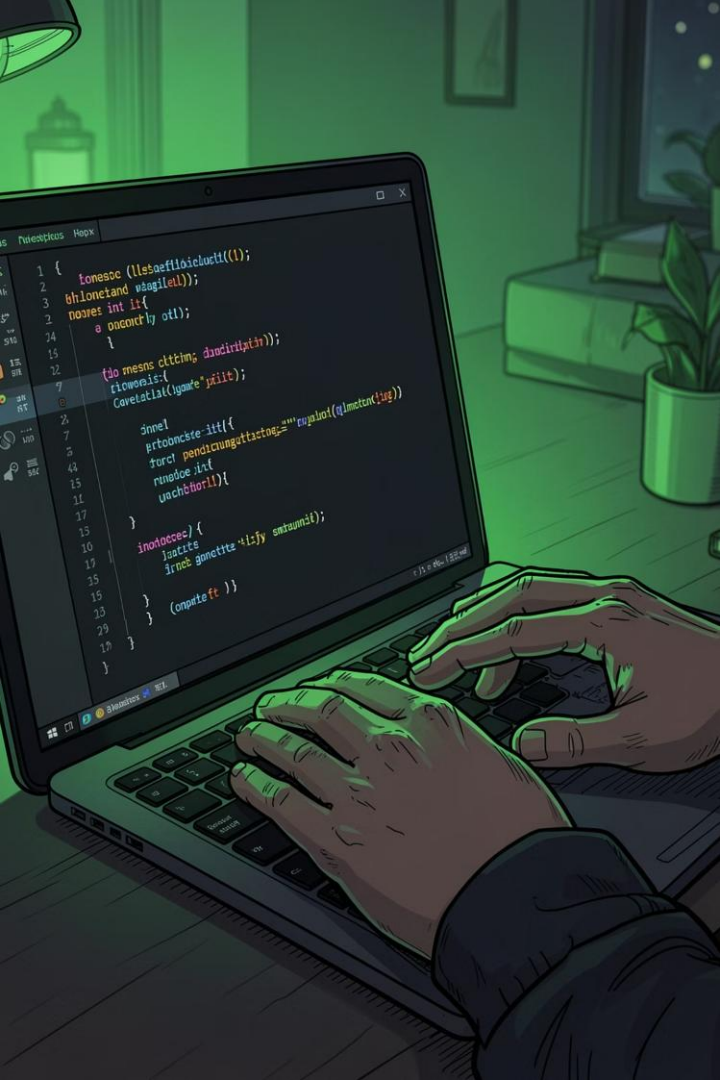


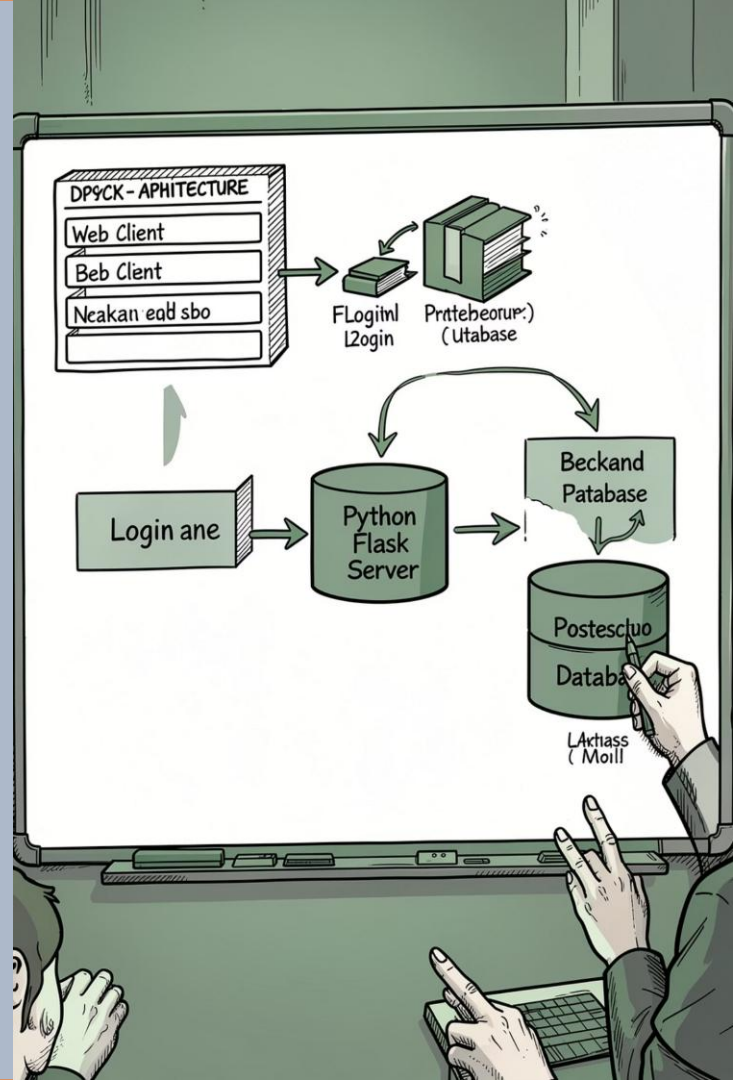


Secure Code Review and Vulnerability Assessment

Focused review of a Python login system: identify vulnerabilities, demonstrate findings from automated and manual analysis, and provide practical remediation and secure-coding guidance for developers and security practitioners.

Project Overview

This assessment targeted a Python-based login system (Flask/Django style).
Scope: authentication endpoints, input handling, database access,
password storage, and configuration. Method: static analysis + manual
review + focused exploit attempts on a sandboxed instance.



Objective & Approach

01

Objective

Uncover security weaknesses affecting authentication, data protection and injection risks.

02

Approach

Combine static analysis (Bandit), dependency checks, manual code inspection, and targeted tests against a development instance.

03

Deliverables

Vulnerability list, exploit examples, remediation steps, and secure-coding best practices tailored for Python web apps.

Tools Used



Bandit

Static analyzer for Python that flags common security issues in source code.



Pytest / Manual Tests

Unit and integration tests to validate input handling and authentication flows.



Manual Code Review

Line-by-line inspection to find logic errors, unsafe SQL patterns and insecure config.

Vulnerabilities Found



SQL Injection

Unsafe string interpolation in SQL queries allowed user-supplied input to alter queries. Risk: data leakage, account takeover, privilege escalation.



Plain-Text Password Storage

Passwords persisted directly to the database without hashing or salting. Risk: mass credential exposure if DB compromised.



No Input Validation

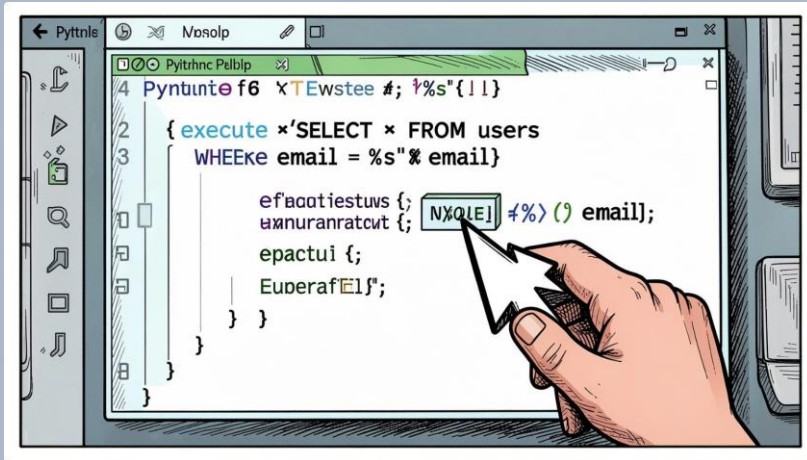
Missing server-side validation on form fields allowed malformed data and increased attack surface for injection and business-logic faults.

Security Analysis — Example Findings

Finding: Direct string formatting used to build SQL queries using user input (e.g. `f"SELECT ... {email}"`).

`cursor.execute(f"SELECT * FROM users WHERE email = '{email}')" # Dangerous: attacker-controlled input can alter the SQL query and lead to data leakage or account takeover.`

- Bandit flagged use of subprocess and format-based DB calls.
- Manual test: payload `" ' OR '1'='1 "` returned additional rows.



Recommendations — Immediate Fixes

Parameterized Queries

Use DB-API parameter binding (e.g. `cursor.execute(query, params)`) or ORM query methods to eliminate injection vectors.

```
query = "SELECT * FROM  
users WHERE email = %s"  
cursor.execute(query,  
[email])
```

Use placeholders instead of string interpolation so user input is never concatenated into SQL.

Password Hashing

Hash passwords with a slow algorithm (bcrypt/argon2) + unique salt. Never store raw passwords or reversible encryption keys in code.

```
import bcrypt  
  
hashed =  
bcrypt.hashpw(password.encode(),  
bcrypt.gensalt())
```

Store only the resulting hash, never the plaintext password.

Server-Side Validation

Implement strict validation and sanitisation for all inputs. Use libraries like marshmallow/pydantic for typed validation and canonicalisation.

```
from pydantic import  
BaseModel, EmailStr  
  
class UserInput(BaseModel):  
    email: EmailStr  
    name: str
```

Validate and normalise data before it reaches your database or business logic.

Recommendations — Secure Coding Practices

Adopt these long-term patterns to reduce future vulnerabilities:

- Principle of least privilege for DB users and app secrets.
- Use prepared statements or ORM parameterisation consistently.
- Protect secrets with managed secret stores (Vault, AWS Secrets Manager) and rotate keys.
- Enforce secure defaults: HTTPS, secure cookies, HSTS, CSP where applicable.
- Integrate SAST/DAST into CI pipelines and block merges on critical findings.

```
import os
```

```
db_password = os.getenv('DB_PASSWORD')
```

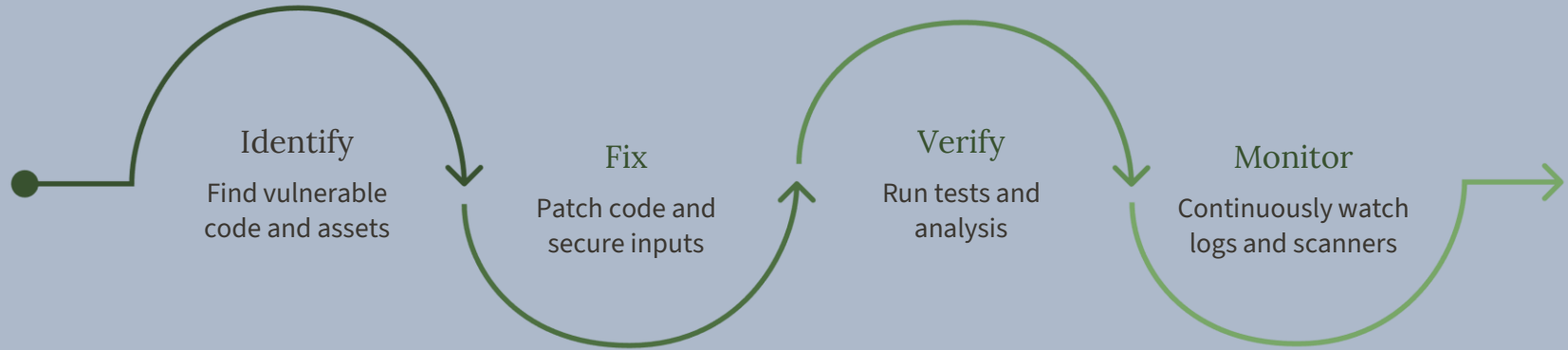
```
user = User.query.filter_by(email=email).first()
```

```
session.permanent = True
```

```
app.config['SESSION_COOKIE_SECURE'] = True
```



Remediation Roadmap



Plan: 1) Patch code (parameterise queries, hash passwords), 2) Run tests and static analysis, 3) Conduct verification testing (unit, integration, fuzzing), 4) Monitor logs and vulnerability scanners continuously.

Conclusion & Next Steps

1

Summary of Findings

Critical issues: SQL injection and plain-text password storage;
medium: missing input validation.
These result in high risk to user data and authentication integrity.

2

Immediate Actions

Apply parameterised queries and strong hashing today; rotate exposed credentials; add server-side validation.

3

Ongoing

Integrate Bandit and dependency checks into CI, schedule regular code reviews, and adopt a secure SDLC with threat modelling for auth flows.



For development teams: prioritise fixes by exploitability and impact. Start with authentication and data protection controls.